PIERCEBench: A Benchmark for Joint Evaluation of Financial Trading Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Evaluating AI trading systems requires accounting for factors beyond their raw trading returns, including whether they use only information available at the time of a decision, how they perform under different market conditions, and the costs of executing and coordinating trades. To address this, we introduce PIERCEBench (Point-in-time Integrity, Impact Aware, Execution Realism, Regime robustness, Coordination value, Evaluation), a four-part evaluation pipeline combining point-in-time historical data, realistic trade execution, and quantitative performance analysis. The data pipeline preserves the information available at each historical timestamp, while the execution replay engine simulates bid-ask spreads, maker/taker fees, slippage, order sequencing, and fill mechanics. The evaluation stage measures temporal integrity, risk-adjusted returns, maximum drawdown, net return, execution-breakeven thresholds, and performance across different market regimes. We apply the pipeline to four backbone models and to TradingAgents, a multi-agent system evaluated with a matched backbone. All evaluated backbones maintained temporal integrity, while trading performance varied across models and market regimes. The multi-agent evaluation additionally accounts for the cost of agent coordination, allowing its performance to be considered alongside the computational cost required to operate it.

1 Introduction

Recent LLM-based trading agents report a wide range of results, with claimed cumulative returns reaching over 400% [18]. From architectures spanning memory-augmented single agents such as FinMem [27], manager-analyst hierarchies such as FinCon [28], and multi-agent debate systems such as TradingAgents [26], AI trading has shifted from static financial QA [8] toward closed-loop and autonomous agents. However, evaluation has not kept pace: time-consistent data splits, transaction-cost models, and execution timing are reported inconsistently, and look-ahead bias [4], backtest overfitting [3], transaction-cost neglect, and regime-shift blindness can each reverse a system's apparent gains. Recent benchmarks such as

StockBench [7] found that returns across 13 LLM trading agents varied widely (from -2.8% to 2.5% over an 82-day window), with performance diverging sharply between downturn and upturn subperiods. AI-Trader [12] found similarly mixed results across U.S. equities, A-shares, and cryptocurrencies: only 3 of 6 agents beat the U.S. QQQ benchmark, none beat the A-share SSE-50, and all six lost money on cryptocurrencies.

Such findings describe individual agents trading alone and offer no evidence on whether combining agents into a coordinating system improves performance or simply adds cost without any benefit (§2). Existing benchmarks do not jointly enforce temporal integrity, execution realism, coordination-cost accounting, and regime-robustness evaluation, so performance differences between systems may

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

reflect the conditions each was tested under rather than differences in design. Evaluation error here carries direct financial costs: a system that appears superior under one market environment can fail once tested against realistic, varied conditions with real execution costs.

We address this gap with PIERCEBench (Point-in-time Integrity, Impact Aware, Execution Realism, Regime robustness, Coordination value, Evaluation), a unified benchmark that evaluates single-agent and multi-agent trading systems under identical data, market regimes, and execution rules. The benchmark jointly measures four properties: whether a system accesses information unavailable at decision time, whether its performance holds after realistic transaction costs, whether it performs consistently across market regimes, and whether its returns reflect skill or general market exposure and movement. Concretely, we build (1) a data pipeline blocking any agent, including communicating agents, from accessing post-decision information; (2) a trading simulator replaying historical fees and slippage that can re-price the same trade log under different cost assumptions; (3) a scoring stage reporting all four properties with confidence intervals across market periods; and (4) a direct test of whether a multi-agent system's advantage survives realistic transaction costs relative to a single-agent baseline.

2 Related Work

Prior evaluations of trading agents typically examine temporal integrity, execution realism, regime robustness, and coordination value separately: of 19 LLM trading-agent studies surveyed by Xia et al. [25], only two used a clean time-based split and one a real cost model.

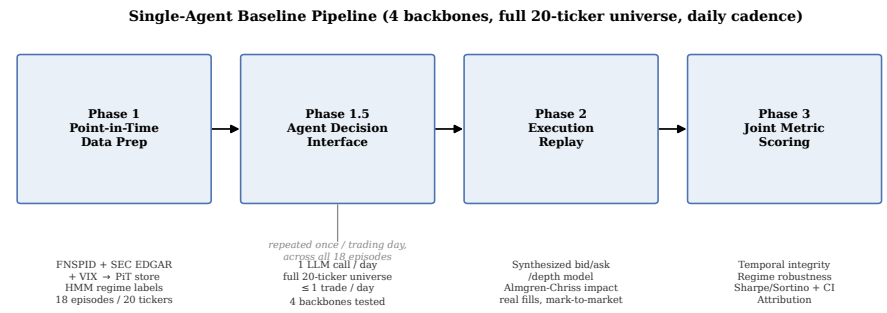
Temporal integrity. Look-Ahead-Bench [4] confirms data leakage (performance drops over 15 points when models shift from training to new data) but lacks a trading simulator, cost model, or coordination test. KTD-Fin [30] instead conceals tickers and dates so a model cannot substitute memory for reasoning. We check every data access, including inter-agent messages, a leak path neither study tests, against a real availability timestamp, auditing every request rather than a sample.

Execution and regime realism. PredictionMarketBench [2] is closest to our simulator, replaying order-book data with fees, but targets prediction markets and single agents, without evaluating coordination. TraderBench [29] stress-tests agents adversarially (8 of 13 models show minimal behavioral change) but holds agent design fixed and reports point estimates without confidence intervals. FINSABER [17] shows that LLM trading advantages from short backtests often vanish over two-decade, 100-plus-stock tests. None of the three replays real fills with a market-impact model, spans calm and crisis regimes with resampled confidence intervals, and tests coordination cost, as we do.

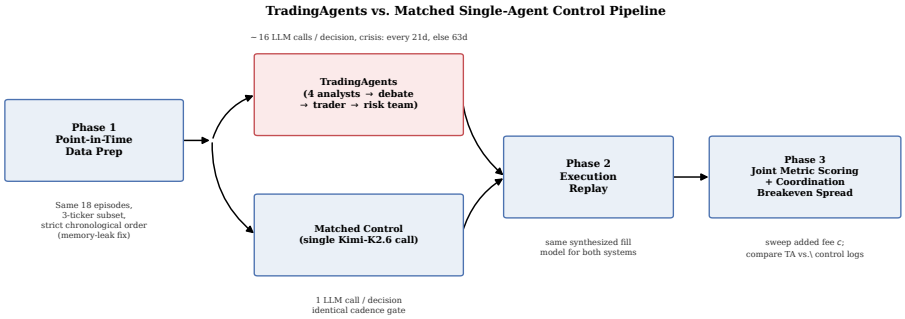
Coordination value. Systems such as TradingAgents [26] show gains from dividing decisions across agents but do not account for coordination's added cost. DeepFund [16] tests live after a model's training cutoff, avoiding look-ahead but still omitting coordination cost. Nguyen and Pham [19] formalize this gap as a Coordination Breakeven Spread but only describe the metric analytically. We implement it by holding a trade log fixed and raising an assumed fee until one system's advantage over another is nullified by its own measured cost.

3 Method

Figure 1 shows how the benchmark works, in four steps. First, we prepare point-in-time market data and label it by market regime (Phase 1). Second, an agent turns each day's data into a trade order (Phase 2). Third, we simulate what would actually happen if that order were placed: the fill price, market impact, and fees (Phase 3). Fourth, we score the results using four measures with confidence intervals (Phase 4). Figure 1(a) shows the basic setup: one AI model looks at data for 20 stocks each day and places one trade. Figure 1(b) shows a comparison: TradingAgents, a system where four AI "analysts" debate and a manager decides, versus a single AI model doing the same job alone, both looking at just 3 stocks. Both setups feed into the same simulation and scoring steps in Phases 3 and 4, so the results can be compared fairly. The only difference between them is whether one AI or several made the trading decision.



(a) Single-agent baseline pipeline (4 backbones, full 20-ticker universe, daily cadence).



(b) TradingAgents versus matched single-agent control pipeline.

Figure 1: The four-phase benchmark pipeline instantiated for (a) the single-agent backbone comparison and (b) the TradingAgents-versus-matched-control comparison. Phases 1, 3, and 4 are shared; only the Phase 2 decision interface and, for the TradingAgents run, the evaluation scope and cadence, differ.

3.1 Point-in-Time Data Preparation (Phase 1)

Every record, prices, fundamentals, news, quotes, corporate actions, carries an available_at timestamp; a query at time t returns only rows with available_at $< t$, and violations are blocked, logged, and counted by the temporal integrity check (§3.5). This same rule also covers inter-agent messages. Cross-agent memory leakage is instead prevented at the episode-scheduling level: TradingAgents' cross-episode memory is protected by running all 18 episodes in true chronological, not regime-grouped, order, so a historically-dated decision can never read a memory entry written for a later episode.

Regimes are labeled by two signals: a walk-forward three-state (trending, mean-reverting, crisis) hidden Markov model [14] over return and realized volatility, refit every 63 days with 8 restarts (2 crisis episodes alone, 2010–2023), and a VIX threshold (close >30 for ≥ 5 days, gaps ≤ 45 days merged; 6 crisis episodes). Episodes are non-overlapping windows separated by a 14-day purge gap, meaning the agent state resets at every episode start.

The design uses 18 episodes (6 per regime, 20 tickers, 2010–2023); each regime totals roughly 370 trading-day observations, enough for stable bootstrap intervals in §3.5. A Fama-French-Carhart six-factor daily series (Mkt, SMB, HML [10], RMW, CMA [11], MOM [6]) and risk-free rate, from the Ken French Data Library [13], are prepared here for §3.5.

3.2 Agent Decision Interface (Phase 2)

Each trading day, the agent receives per ticker: prior close, 5/21/63-day returns, realized volatility, fundamentals (leverage, current ratio, return on assets), and a news-item count, preserving volume as signal while removing a prompt-injection surface, plus its own cash, positions, and 10-day trade history. The regime label itself is withheld, so the test measures whether a strategy survives an unlabeled shift rather than whether the agent can read a label; realized volatility, an HMM input feature, stays visible.

The agent returns one decision per day (action, symbol, quantity, order type, limit price where applicable) across three order types: market, limit, and post-only, which is rejected rather than filled immediately, so it only fills by waiting. Each decision is validated: the symbol must be in the universe, sell quantity is clipped to the position actually held, and buy quantity is clipped by cash affordability at the pre-decision price; For example, a ticker with no price data that day will fall back to a hold rather than aborting the episode.

The loop runs unmodified across three backends (native Anthropic API, Azure AI Foundry, and Azure Marketplace), retrying with capped exponential backoff on transient failures only; token usage and latency are measured per call, feeding §3.5's coordination-cost accounting.

3.3 TradingAgents Decision Interface

The TradingAgents baseline uses the same execution environment and accounting framework, with decision-making replaced by the TradingAgents multi-agent system [26]: four analyst roles (fundamentals, sentiment, news, market) each produce an independent read; a bull and bear researcher debate those reads; a research manager resolves the debate; a trader proposes a position; and a three-member risk team debates it before a portfolio manager issues a final, deterministically-mapped rebalancing target. Because the full pipeline costs substantially more per decision than a single model call, TradingAgents' ticker scope and decision cadence are reduced for this study (§4); its decisions pass through the same validation and execution process as the single-agent baselines, with the portfolio marked to market daily between consultations. This reduced scope would otherwise confound coordination with decision frequency and universe size, so TradingAgents is compared only against a matched single-agent Kimi-K2.6 control run under the identical scope and cadence.

3.4 Execution Replay (Phase 3)

Each validated order fills against a synthesized bid/ask/depth model (§7): bid and ask derive from that day's closing price under a fixed 5-basis-point spread, with depth capped at 2% of trailing average daily volume, applied identically across all 18 episodes. We model both spread and depth limit to reflect realistic cost compared to idealized fills. Orders that consume liquidity, market and marketable limit orders, cross this spread and pay a taker fee; orders that supply it, post-only and resting limit orders, instead wait to be crossed and receive a rebated maker fee. This helps enforce the asymmetric cost of demanding versus supplying liquidity in real markets.

Taker fills also incur market impact, priced with the square-root temporary-impact model of Almgren and Chriss [1]:

$$p_{exec} = p_{touch} \times \left(1 \pm \gamma \cdot \sigma \cdot \sqrt{q/ADV}\right)$$

where p_{touch} is the best bid or ask at the fill, γ the impact coefficient, σ realized volatility, q order size, and ADV average daily volume. This pushes the execution price away from the quoted price by an amount that grows with order size relative to typical volume and with volatility, in the direction of the trade. We add this because in real trading, larger or more volatile trades move prices and only liquidity-demanding orders pay for it. Impact applies only to taker fills, since a resting order supplies rather than consumes depth. Three constraints apply at fill time to further reflect conditions which would realistically occur: quantity is capped at available depth, a buy cannot exceed cash on hand once the post-impact, post-fee price is known (a check Phase 2 cannot make in advance), and a limit order never executes worse than its stated limit; fees are charged on the post-slippage notional. The portfolio is marked to that day's close daily, avoiding a one-day lag.

3.5 Joint Metric Scoring (Phase 4)

All four properties are computed from one execution history so that the differences between our reported numbers reflect differences in behavior rather than differences between runs. Temporal integrity passes if the access log shows zero flagged violations across every episode. Regime robustness reports per-episode Sharpe, Sortino, net return, and max drawdown grouped by regime as mean $\pm$ std, so a regime-specific collapse is not averaged away.

With r_t the simulated return on day t and r_f the matching risk-free rate, the Sharpe ratio [22] is:

$$\text{Sharpe} = \left(\frac{\text{mean}(r_t - r_f)}{\text{std}(r_t - r_f)} \right) \times \sqrt{252}$$

$$\text{Sortino} = \left(\frac{\text{mean}(r_t - r_f)}{\sqrt{\frac{1}{N} \sum_t \min(r_t - r_f, 0)^2}} \right) \times \sqrt{252}$$

Sortino follows Sortino and Price [23]: the denominator is the downside semi-deviation over all N observations (positive-return days set to zero, not dropped). Both ratios use simulated returns from actual fills.

Factor attribution measures if positive returns are skill or exposure to a well-known pattern in how markets move, as a strategy which holds small, cheap, or trending stocks can look skilled without actually being so. Attribution regresses excess returns on the six factors defined in §3.1:

$$r_t - r_f = \alpha + \beta_{mkt} \cdot Mkt_t + \beta_{smb} \cdot SMB_t + \beta_{hml} \cdot HML_t + \beta_{rmw} \cdot RMW_t + \beta_{cma} \cdot CMA_t + \beta_{mom} \cdot MOM_t + \varepsilon_t$$

reported at CAPM [21], three-factor [10], and six-factor [6, 11] specifications, so a shrinking alpha as factors are added flags exposure to a known return source rather than skill; a positive six-factor alpha indicates returns unexplained by any included factor, not proof of skill by itself.

95% confidence intervals for Sharpe, Sortino, and alpha use a circular moving-block bootstrap [15, 20]: 1,000 resamples of contiguous 21-day blocks drawn with wraparound so edge days are sampled as often as interior ones, preserving the short-range autocorrelation in daily returns that independent resampling would remove and otherwise understate.

Coordination value measures how large a per-trade fee would a system's coordination value need to be to justify not using a simpler alternative. Coordination value sweeps an added fee c upward from zero, holding trade history, costs, and regime labels fixed:

$$c^* = \min\{c \geq 0 : \text{CumulativeAdvantage}(c) \leq 0\}$$

where CumulativeAdvantage(c) is the difference in cumulative net return between the two systems after re-charging every logged fill at rate c (re-scoring Phase 3's log rather than re-simulating). An advantage surviving a large c is robust to cost assumptions; one vanishing near $c = 0$ is specific to the original run's costs. Both systems must share an identical episode set, verified before comparison. A companion accounting layer charges the comparison system for coordination overhead (token cost plus a latency-driven signal-decay penalty), so the advantage is net of the cost to produce it.

4 Experimental Setup

Table 1 summarizes the benchmark's scale and construction before we walk through each component below.

Data Sources Three sources feed the point-in-time store, each timestamped and gated separately. Prices and news come from FNSPID [9] (daily OHLCV paired with time-stamped news across roughly 4,800 S&P 500 companies, 1999–2023); each article's publication timestamp is its available_at value. Fundamentals come from SEC EDGAR [24] as structured XBRL facts (total liabilities, total assets, current assets/liabilities, net income), timestamped by filing acceptance date,

Table 1: PIERCEBench summary statistics.

Property	Value
Tradable universe	20 equities
Date range	2010–2023
Data sources	FNSPID (price/news), Alpaca (quotes), SEC EDGAR (fundamentals), Ken French Library (factors)
Episodes	18 (6 trending, 6 mean-reverting, 6 crisis)
Avg. episode length	63 trading days
Inter-episode purge gap	14 days
Starting capital per episode	$100,000
Single-agent backbones evaluated (full scope)	4 (Claude Opus 5, GPT-5.6-Sol, DeepSeek-V4-Pro, Kimi-K2.6)
Single-agent decisions (full scope, per backbone)	∼1,126 trading-day decisions
TradingAgents matched-control scope	3 tickers, 32 decision-days, 94 ticker-decisions
Bootstrap resamples (block length)	1,000 (21 trading days)

typically weeks to months after quarter-end, rather than fiscal period; derived ratios (leverage, current ratio, return on assets) use only the most recent as-filed facts known as of the decision date, never as restated values. Execution-level detail (the bid/ask/depth model driving Phase 3 fills) is synthesized from each day's closing price under a fixed spread and volume-based depth cap; finer intraday Alpaca quotes are collected separately from 2016 onward but feed only the coordination-cost calculation.

Evaluation Universe and Episodes The tradable universe comprises 20 equities: AAPL, MSFT, GOOG, AMZN, INTC, NVDA, TSLA, JPM, V, JNJ, WMT, CL, XOM, ABBV, LOW, AXP, DIS, WFC, KO, PFE. SPY, GLD, and FXI serve only as a passive-return comparator, never as tradable instruments or fundamentals sources (ETFs file no SEC periodic reports and receive no CIK). SPY additionally defines the trading calendar and is the sole input to the regime-labeling procedure of §3.1. Episodes are constructed by labeling each trading day via the HMM/VIX procedure of §3.1, then sampling 18 non-overlapping episodes (6 per regime, averaging 63 trading days, 2010–2023), each separated by the 14-day purge gap described there. Every episode begins with $100,000 cash and zero holdings.

Single-Agent Baselines Each of the four backbones (Claude Opus 5, GPT-5.6-Sol, DeepSeek-V4-Pro, Kimi-K2.6) runs the single-agent decision loop of §3.2: one model call per trading day, deciding across the full 20-ticker universe, at most one trade per day, evaluated at full daily cadence across all 18 episodes (roughly 1,126 trading-day decisions). Measured inference cost was $30.34 (Claude Opus 5), $23.47 (GPT-5.6-Sol), $6.92 (DeepSeek-V4-Pro), and $15.61 (Kimi-K2.6).

TradingAgents versus a Matched Single-Agent Control TradingAgents [26] (architecture in §3.3) cannot be evaluated under the same conditions as the single-agent baselines. During integration we found that its fundamentals vendor ignores the curr_date parameter and always returns the company's current, live snapshot regardless of simulated date, which would have been an unconditional look-ahead leak. We replaced it with a drop-in vendor backed by our own point-in-time store, enforcing the same available_at binding used throughout our pipeline.

TradingAgents issues roughly 16 real model calls per single-ticker decision against one for a single-agent baseline; at full scope (20 tickers, daily cadence, 18 episodes) this prices a run at thousands of dollars per backbone, unfeasible for our budget. Two reductions follow: the ticker universe is cut to the first 3 of the 20-ticker list (AAPL, MSFT, GOOG), and cadence becomes asymmetric and regime-conditioned (holding position on non-decision days while still marking to market daily): one real decision every 21 trading days in crisis episodes and every 63 otherwise, concentrated on crisis episodes since the budget could not support dense sampling across all three regimes. The regime label itself is withheld, so this cadence choice does not tell the system which regime it is in.

Both TradingAgents and its control run the same 18 episodes, dates, and regime labels as the baselines, but in true chronological rather than regime-grouped order (crisis, then trending, then mean-reverting, as otherwise stored), because TradingAgents' cross-episode self-reflection memory injects its own past decisions into future prompts; For example, in regime-grouped order a 2010-dated decision could read memory already written for a 2022-dated crisis episode, a leak the standard point-in-time check does not audit since it covers only the price, news, and fundamentals loaders. We confirmed this as an actual leak in a completed run's memory log and fixed it by sorting episodes by start date. The control is the same Kimi-K2.6 backbone powering TradingAgents' roles, run as a single-agent baseline under this identical universe, episode order, and schedule, isolating coordina-

tion from decision frequency; the Coordination Breakeven Spread is computed against it rather than the full-daily, full-universe Kimi-K2.6 baseline, since comparing reduced-cadence TradingAgents to a daily-deciding baseline would confound coordination with frequency.

5 Results

We report single-agent backbone comparison at full scope (5.1), then isolate the effect of TradingAgents' coordination pipeline against a matched single-agent control (5.2).

5.1 Single-Agent Backbone Comparison

Table 2: Aggregate Executable Performance across the four single-agent backbones.

Backbone	Temporal Integrity	Sharpe (95% CI)	Sortino (95% CI)	Max Drawdown	Net Return	Inference Cost
Claude Opus 5	PASS	1.720 [0.971, 2.624]	2.580 [1.380, 4.219]	6.69%	\$9,347	\$30.34
Kimi-K2.6	PASS	1.407 [0.566, 2.309]	2.161 [0.822, 3.785]	7.95%	\$8,962	\$15.61
GPT-5.6-Sol	PASS	0.954 [0.128, 1.856]	1.315 [0.171, 2.745]	5.83%	\$3,323	\$23.47
DeepSeek-V4-Pro	PASS	−0.819 [−1.697, 0.272]	−0.993 [−1.877, 0.415]	6.37%	−\$2,502	\$6.92

Table 2 reports aggregate Executable Performance across all four single-agent backbones, each run at full daily cadence across the full 20-ticker universe and all 18 episodes; Max Drawdown and Net Return are equal-weighted averages across the three regime-level means, exact here since each regime contains exactly 6 episodes.

All four backbones pass Temporal Integrity, with zero flagged access-log violations across every episode. A separate per-backbone breakeven-fee diagnostic returns 0 basis points for all four and is not reported in Table 2: it sums each episode's signed fill notional (sells positive, buys negative) and searches for the per-trade fee at which that sum crosses zero, which only happens if a strategy closes every position it opens (a round trip); a backbone that ends an episode still holding a position it opened, common in trending and crisis regimes, never produces a crossing regardless of profitability. Net Return in Table 2, computed from actual portfolio value rather than fill notional, does not share this limitation and is positive for three of the four backbones.

Three of the four backbones (Opus, Kimi, GPT-5.6-Sol) have Sharpe CIs excluding zero, a statistically distinguishable positive result; DeepSeek-V4-Pro's CI spans $[-1.70, 0.27]$, crossing zero, so its negative point estimate is not distinguishable from a null result at 95% confidence, consistent with its regime-level performance below.

Only Claude Opus 5 and Kimi-K2.6 have six-factor alpha CIs excluding zero (0.094–0.307 and 0.0095–0.411), returns unexplained by market, size, value, profitability, investment, or momentum factors; both stay stable across the CAPM→3-factor→6-factor ablation rather than shrinking toward zero (Opus: 0.249→0.235→0.201; Kimi: 0.251→0.225→0.199), so the signal is not an artifact of under-specifying the factor model. GPT-5.6-Sol's $[-0.052, 0.142]$ and DeepSeek's $[-0.324, 0.010]$ both cross zero, so GPT-5.6-Sol's positive Sharpe point estimate is not distinguishable from factor exposure at this sample size.

Regime-level results diverge from the aggregate ranking (Figure 2): ranked by full-sample Sharpe, Opus leads, but stratified by regime Kimi-K2.6 posts the highest raw net return of any backbone in crisis specifically (\$12,929 vs. Opus's \$10,857), while trailing Opus in trending (\$9,990 vs. \$12,208) and mean-reverting (\$3,968 vs. \$4,978). DeepSeek-V4-Pro's negative returns concentrate in trending and mean-reverting (−\$5,824, −\$2,593) rather than crisis, where it posts a small positive net return (\$912), a pattern its aggregate Sharpe of -0.819 does not show.

Cost and performance are not monotonically related (Figure 2). DeepSeek-V4-Pro is both the cheapest backbone tested (\$6.92) and the only one with a negative Sharpe point estimate and negative Net Return (−\$2,502). Kimi-K2.6, at roughly half Opus's cost (\$15.61 vs. \$30.34), reaches 82% of Opus's Sharpe point estimate and has an alpha CI that, like Opus's, excludes zero. Spending more on GPT-5.6-Sol (\$23.47) did not produce a higher Sharpe point estimate than spending less on Kimi.

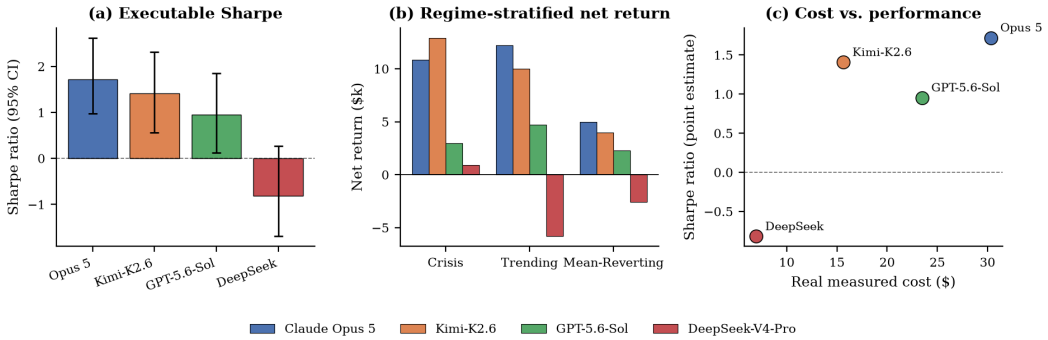


Figure 2: Aggregate Executable Performance across the four single-agent backbones: (a) Sharpe ratio with 95% CIs, (b) regime-stratified net return, and (c) real measured cost vs. Sharpe ratio point estimate.

5.2 TradingAgents versus a Matched Single-Agent Control

This comparison isolates TradingAgents' coordination pipeline from backbone choice, ticker scope, or decision cadence: both systems share the same Kimi-K2.6 backbone, 3-ticker universe, 18 episodes in identical chronological order, and per-regime decision cadence (§4); the only variable is whether one model decides alone or four analyst roles, a bull/bear debate, a research manager, a trader, and a three-member risk team decide together.

Table 3: TradingAgents versus a matched single-agent Kimi-K2.6 control.

System	Sharpe (95% CI)	Sortino (95% CI)	6-factor α (95% CI)	Total Cost
Matched-Control Kimi-K2.6	1.020 [0.293, 1.758]	1.561 [0.417, 2.953]	0.0857 [−0.072, 0.227]	$0.34
TradingAgents	0.650 [−0.187, 1.450]	0.943 [−0.257, 2.246]	−0.0032 [−0.018, 0.009]	$33.41

In Table 3, the matched control's Sharpe and Sortino CIs both exclude zero, a statistically distinguishable positive result; TradingAgents' do not (Sharpe CI [−0.187, 1.450], Sortino CI [−0.257, 2.246], both crossing zero), so its performance is not distinguishable from a null result at this sample size. We did not run a paired significance test between the two systems' Sharpe ratios directly, so this alone does not establish the control's Sharpe is significantly higher; what it establishes is that the control's point estimate leads on every metric and in every regime reported here, and only its estimate clears the zero threshold.

Regime-level net returns show the same direction throughout (Figure 3): the matched control outreturns TradingAgents by roughly 14x in crisis ($4,177 vs. $296), 15x in trending ($9,074 vs. $599), and 6x in mean-reverting ($3,929 vs. $667). Neither alpha CI excludes zero; the control's positive point estimate (0.0857, CI [−0.072, 0.227]) is not by itself evidence of significant skill, though its sign agrees with the Sharpe, Sortino, and net-return advantage above, while TradingAgents' alpha point estimate is close to zero either way ([−0.018, 0.009]).

Coordination cost, charged. TradingAgents made 94 real ticker-decisions across 32 decision-days over its 18 episodes, against the matched control's 32 decisions, a real measured cost of $33.41 versus $0.34, roughly 98x, reflecting real coordination overhead (roughly 16 LLM calls per ticker-decision against 1) rather than a difference in scope or cadence. Sweeping an added fee upward from the systems' real logged costs, the Coordination Breakeven Spread is 0 basis points in both directions (trading costs only, and with the full coordination bill charged): no cost level, however small, would have made TradingAgents' pipeline worth running instead of the single-agent control here, since the control's point estimate already leads in every regime before any coordination cost is applied.

Scope of this comparison. This result is specific to a 3-ticker universe, Kimi-K2.6 as the shared backbone, and a crisis-concentrated 21/63-day cadence, not a general claim about TradingAgents'

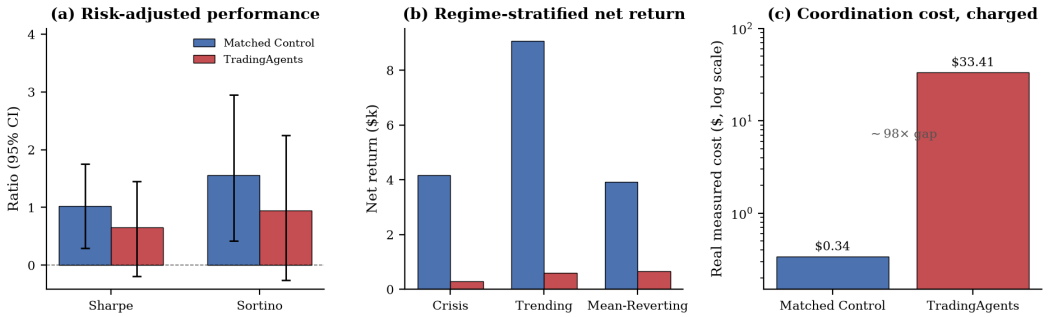


Figure 3: TradingAgents versus the matched single-agent Kimi-K2.6 control: (a) risk-adjusted performance with 95% CIs, (b) regime-stratified net return, and (c) real measured coordination cost on a log scale.

architecture at full ticker scope, daily cadence, or with a different backbone; §7 discusses how these scope choices were forced by real measured cost (§4) and what full-scope testing would require.

6 Conclusion

This project set out to answer two questions about LLM-driven trading systems: how much of reported performance is attributable to backbone choice rather than system design, and whether a multi-agent coordination pipeline earns back its real dollar cost once that cost is charged against its performance rather than assumed away. Both required point-in-time-safe data, realistic execution, and measured API cost, none of which is standard practice in the broader LLM-trading-agent literature (§2); the findings are precise within the conditions tested, not general claims about backbone selection or multi-agent architectures beyond them.

Backbone choice determined trading edge, and regime-level results diverged from the aggregate ranking, underscoring that a single full-sample Sharpe number can hide which backbone wins under a given market regime (§5.1). The central result concerns coordination value: holding backbone, universe, and cadence fixed against a matched single-agent control, TradingAgents' coordination pipeline did not earn back its cost under these scoped, budget-constrained conditions (§5.2, §7).

Both findings depend on measuring cost directly rather than assuming it: the cost figures are what separate the most cost-efficient backbone from a more expensive, weaker one, and TradingAgents underperforms the matched control not because of its architecture or debate structure as such, but because its real measured cost, set against its real measured performance gap, produces a Coordination Breakeven Spread of zero, which is a result invisible from Sharpe ratios alone.

7 Limitations

The 20-ticker universe is fixed rather than reconstructed for each historical date, so firms that were prominent during 2010–2023 but were later delisted, acquired, or failed are excluded [5]. Furthermore, Trade fills are priced from each day's closing price under a fixed spread and a volume-based depth cap, not real intraday quotes; real quotes exist but only from 2016 onward and feed only the coordination-cost calculation, not the fill engine. No episode covers 2008, since SEC EDGAR's structured fundamentals data starts in 2009 Q1, leaving the benchmark unable to capture the major stock market crash and bear market of 2008.

TradingAgents is evaluated on 3 tickers at a reduced, regime-conditioned cadence rather than the full universe and daily cadence used for the single-agent baselines, since full-scope evaluation would cost an estimated $8,000–$9,800 per backbone, which is unfeasible considering our budget; the valid coordination comparison is against a matched single-agent baseline under this same restricted scope, not the full-scope baselines, and is priced at current rates for one model on one hosting route, which billed differently than the same model's other routes during this project. Not every part of TradingAgents was audited for temporal leaks: one leak was found and fixed in its fundamentals loader, but news retrieval was only spot-checked and its filing-date handling was not confirmed.

References

[1] Almgren, R., & Chriss, N. (2000). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5–39.

[2] Arora, A., & Malpani, R. (2026). PredictionMarketBench: A SWE-bench-style framework for backtesting trading agents on prediction markets. *arXiv:2602.00133*.

[3] Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5), 458–471.

[4] Benhenda, M. (2026). Look-Ahead-Bench: A standardized benchmark of look-ahead bias in point-in-time LLMs for finance. *arXiv:2601.13770*.

[5] Brown, S. J., Goetzmann, W., Ibbotson, R. G., & Ross, S. A. (1992). Survivorship bias in performance studies. *The Review of Financial Studies*, 5(4), 553–580.

[6] Carhart, M. M. (1997). On persistence in mutual fund performance. *The Journal of Finance*, 52(1), 57–82.

[7] Chen, Y., Yao, Z., Liu, Y., Xin, A., Ye, J., Yu, J., Hou, L., & Li, J. (2025). StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv:2510.02209*.

[8] Chen, Z., Chen, W., Smiley, C., Shah, S., Borova, I., Langdon, D., Moussa, R., Beane, M., Huang, T.-H., Routledge, B., & Wang, W. Y. (2021). FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP 2021* (pp. 3697–3711).

[9] Dong, Z., Fan, X., & Peng, Z. (2024). FNSPID: A comprehensive financial news dataset in time series. *arXiv:2402.06698*.

[10] Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1), 3–56.

[11] Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics*, 116(1), 1–22.

[12] Fan, T., Yang, Y., Jiang, Y., Zhang, Y., Chen, Y., & Huang, C. (2025). AI-Trader: Benchmarking autonomous agents in real-time financial markets. *arXiv:2512.10971*.

[13] French, K. R. (n.d.). *Kenneth R. French Data Library*. Tuck School of Business, Dartmouth College.

[14] Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2), 357–384.

[15] Künsch, H. R. (1989). The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3), 1217–1241.

[16] Li, C., Shi, Y., Wang, C., Duan, Q., Ruan, R., Huang, W., Long, H., Huang, L., Tang, N., & Luo, Y. (2025). Time travel is cheating: Going live with DeepFund for real-time fund investment benchmarking. *arXiv:2505.11065*.

[17] Li, W. W., Kim, H., Cucuringu, M., & Ma, T. (2025). Can LLM-based financial investing strategies outperform the market in long run? *arXiv:2505.07078*.

[18] Li, X., Zeng, Y., Xing, X., Xu, J., & Xu, X. (2025). HedgeAgents: A balanced-aware multi-agent financial trading system. In *Companion Proceedings of the ACM Web Conference 2025*. *arXiv:2502.13165*.

[19] Nguyen, P., & Pham, T. (2026). Toward reliable evaluation of LLM-based financial multi-agent systems: Taxonomy, coordination primacy, and cost awareness. *arXiv:2603.27539*.

[20] Politis, D. N., & Romano, J. P. (1992). A circular block-resampling procedure for stationary data. In *Exploring the Limits of Bootstrap* (pp. 263–270). Wiley.

[21] Sharpe, W. F. (1964). Capital asset prices: A theory of market equilibrium under conditions of risk. *The Journal of Finance*, 19(3), 425–442.

[22] Sharpe, W. F. (1966). Mutual fund performance. *The Journal of Business*, 39(S1), 119–138.

[23] Sortino, F. A., & Price, L. N. (1994). Performance measurement in a downside risk framework. *The Journal of Investing*, 3(3), 59–64.

[24] U.S. Securities and Exchange Commission. (n.d.). *Financial Statement Data Sets*.

[25] Xia, Y., You, P., Wang, T., Liu, F., Qi, H., Wu, X., & Zhang, S. (2026). Agentic trading: When LLM agents meet financial markets. *arXiv:2605.19337*.

[26] Xiao, Y., Sun, E., Luo, D., & Wang, W. (2024). TradingAgents: Multi-agents LLM financial trading framework. *arXiv:2412.20138*.

[27] Yu, Y., Li, H., Chen, Z., Jiang, Y., Li, Y., Zhang, D., Liu, R., Suchow, J. W., & Khashanah, K. (2023). FinMem: A performance-enhanced LLM trading agent with layered memory and character design. *arXiv:2311.13743*.

[28] Yu, Y., Yao, Z., Li, H., Deng, Z., Cao, Y., Chen, Z., Suchow, J. W., Liu, R., Cui, Z., Xu, Z., Zhang, D., Subbalakshmi, K., Xiong, G., He, Y., Huang, J., Li, D., & Xie, Q. (2024). FinCon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *NeurIPS 2024*. *arXiv:2407.06567*.

[29] Yuan, X., Xu, H., Xu, S., Zou, C., & Xiong, J. (2026). TraderBench: How robust are AI agents in adversarial capital markets? *arXiv:2603.00285*.

[30] Zhu, T., Zhao, W., Sun, R., Luan, B., Lu, J., Wang, S., Li, J., Jiang, D., He, Y., & Bai, Z. (2026). From knowing to doing: A memory-controlled benchmark for LLM trading agents on stock markets. *arXiv:2605.28359*.